

Software Testing and Quality Assurance

Exam Answer Guide

Mumbai University · BSc CS · Semester 5

15 most frequently asked questions, with diagrams, tables and worked examples

Contents

1. Q1. Quality, Software Testing, Quality Assurance and software quality factors
2. Q2. Errors, types of errors, defects and the Defect Management Process
3. Q3. Role of testing in each SDLC phase and the V-Model
4. Q4. Software Review, Inspection and Walkthrough; Verification vs Validation
5. Q5. Software Testing Metrics, their types and the metrics life cycle
6. Q6. Black Box vs White Box testing and their techniques
7. Q7. Integration testing approaches: Top-down and Bottom-up
8. Q8. System testing, its types, and Alpha and Beta testing
9. Q9. Smoke testing and its benefits
10. Q10. Cause and effect diagram, Pareto diagram and Scatter diagram
11. Q11. Cyclomatic Complexity with an example
12. Q12. Quality Costs: types and how to measure them
13. Q13. Software reliability: elements, factors and metrics
14. Q14. Guidelines for Formal Technical Reviews (FTR)
15. Q15. Six Sigma (DMAIC) and ISO 9000 standards

Q1. Quality, Software Testing, Quality Assurance and software quality factors

Definitions

Quality

- IEEE: *"the degree to which a system, component or process meets specified requirements, and customer or user needs or expectations."*
- In simple words, quality means **fitness for use** (Juran) and **conformance to requirements** (Crosby).
- Quality has two sides:
 - **Quality of design**: the features the designers put in the product.
 - **Quality of conformance**: how closely the finished product follows that design.

Software Testing

- Testing is *the process of executing a program with the intent of finding errors* (Glenford Myers).
- It checks whether the actual results match the expected results, so that the software is as defect-free as possible.
- **Goals of testing**: find defects early, check that requirements are met, build confidence in the product, and lower the risk of failure.
- Testing can show that defects are present. It cannot prove that there are none.

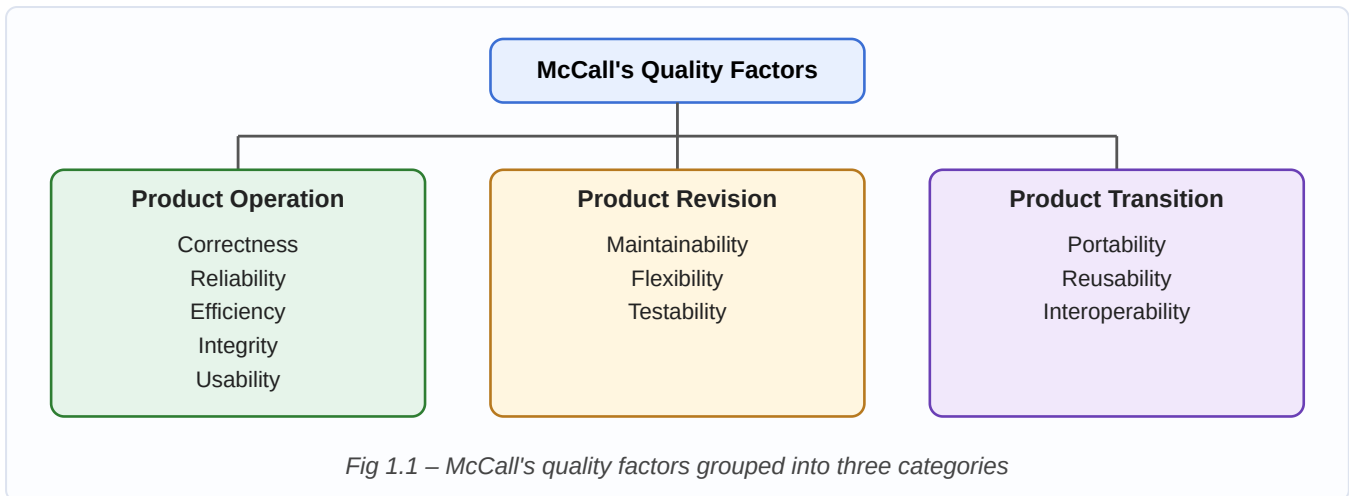
Quality Assurance (QA)

- QA is a **planned and systematic set of activities** that gives confidence that the software *process* will produce a product of the required quality.
- QA is **process-oriented and preventive**: it sets standards, audits, reviews and process improvement.
- Testing and Quality Control (QC) are **product-oriented and detective**: they find defects in the product.

Quality Assurance (QA)	Quality Control (QC) / Testing
Focuses on the process	Focuses on the product
Prevents defects	Detects defects
Proactive	Reactive
Done throughout the SDLC	Done after the product or part of it is built
Examples: audits, standards, process definition	Examples: testing, inspections of the product

Software quality factors (McCall's model, 1977)

McCall groups **11 quality factors** into three views of the product:



Category	Factor	Meaning
Product Operation (how it runs)	Correctness	Does it do what the specification says?
	Reliability	Does it do it accurately every time, without failure?
	Efficiency	Does it use CPU, memory and time economically?
	Integrity	Is it secure from unauthorised access?
	Usability	Is it easy to learn and use?
Product Revision (how easily it changes)	Maintainability	How easily can bugs be found and fixed?
	Flexibility	How easily can it be modified or enhanced?
	Testability	How easily can it be tested?
Product Transition (how it adapts)	Portability	Can it run on another platform?
	Reusability	Can parts be reused in other programs?
	Interoperability	Can it work together with other systems?

ISO 9126 / ISO 25010 gives a similar list of quality characteristics: Functionality, Reliability, Usability, Efficiency, Maintainability and Portability.

Q2. Errors, types of errors, defects and the Defect Management Process

What is an error?

- **Error (mistake):** a human action that produces an incorrect result, for example a misread requirement or a typing slip.
- **Fault / Defect / Bug:** the result of the error, recorded in the code or in a document.
- **Failure:** what happens when the faulty code runs and the system behaves differently from what is expected.

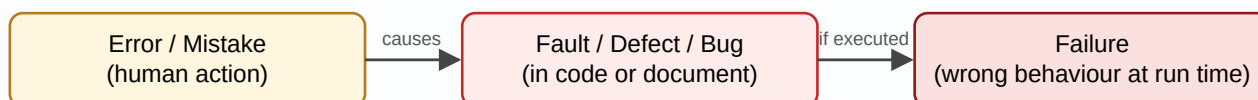


Fig 2.1 – Error → Fault (Defect) → Failure

Types of errors

Type	Description / example
Requirement errors	Missing, ambiguous or wrong requirements. These are the most expensive to fix late.
Design errors	Wrong algorithm, wrong data structure, poor interface design.
Syntax errors	The code breaks the language rules, for example a missing semicolon. The compiler catches these.
Logical errors	The program runs but gives wrong output, for example > used instead of >=.
Calculation errors	Wrong formula, wrong rounding or a type overflow.
Runtime errors	The program crashes while running, for example divide by zero or a null pointer.
Interface / integration errors	Modules pass data wrongly, for example a parameter mismatch.
Boundary-related errors	Wrong handling at the edges of a range, such as the first or last element of an array.
Control-flow errors	Wrong loop or branch, for example an infinite loop.
Error-handling errors	Exceptions are not caught, or the error messages are unclear.
User-interface errors	Wrong labels, broken navigation, poor usability.
Documentation errors	The manual or help does not match the software.
Load / race-condition errors	Failures that appear only under heavy load or with concurrent access.

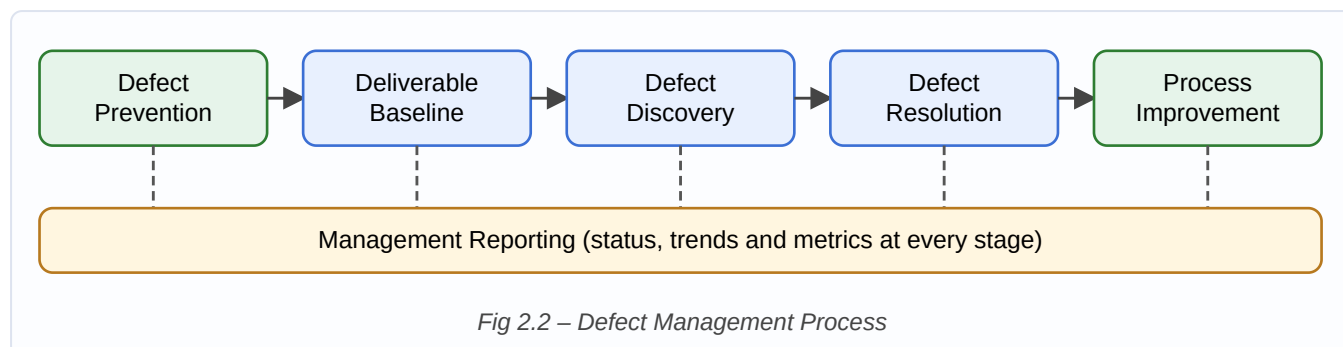
Defect

- A **defect** is a variance between the expected and the actual result. It may be:
 - a **wrong** implementation of a requirement,
 - a **missing** requirement, or
 - an **extra** feature that was never specified.

- **Severity** shows how badly the defect affects the system: Critical, Major, Minor or Trivial.
- **Priority** shows how soon the defect must be fixed: High, Medium or Low.

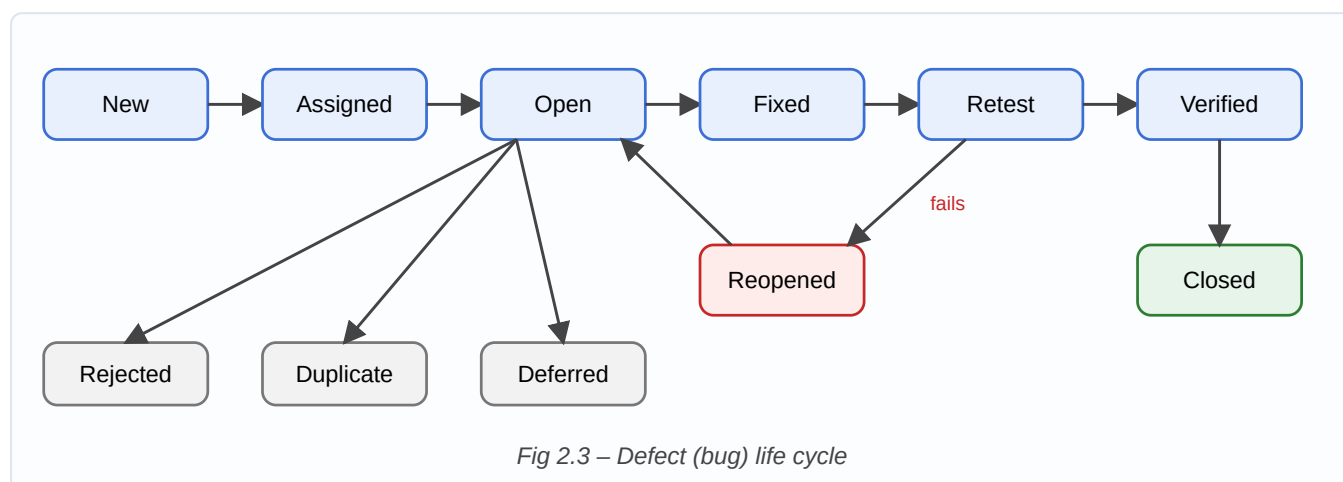
Defect Management Process (DMP)

The aim of the DMP is to **prevent defects, find them early, reduce their impact and improve the process.**



1. **Defect Prevention:** identify critical risks, estimate their impact and reduce them using standards, training and reviews.
2. **Deliverable Baseline:** a deliverable (document, code, build) is formally frozen. Any defect found after this point is formally tracked.
3. **Defect Discovery:** find defects through reviews and testing, report them, and get the developer to acknowledge them as valid.
4. **Defect Resolution:** developers prioritise, fix and verify the defect, and the tester is informed.
5. **Process Improvement:** analyse why each defect happened and improve the process so that similar defects do not happen again.
6. **Management Reporting:** throughout the process, defect data is reported to management (counts, trends, ageing) to support decisions.

Defect life cycle



State	What it means
New	The defect is logged for the first time.
Assigned	It is given to a developer.
Open	The developer is analysing or fixing it. From here it may also be marked Rejected (not a bug), Duplicate or Deferred (fix in a later release).

State	What it means
Fixed	The code change is made.
Retest	The tester re-tests the fix.
Verified	The fix works.
Closed	The defect is finished.
Reopened	The fix did not work, so the defect goes back to the developer.

Q3. Role of testing in each SDLC phase and the V-Model

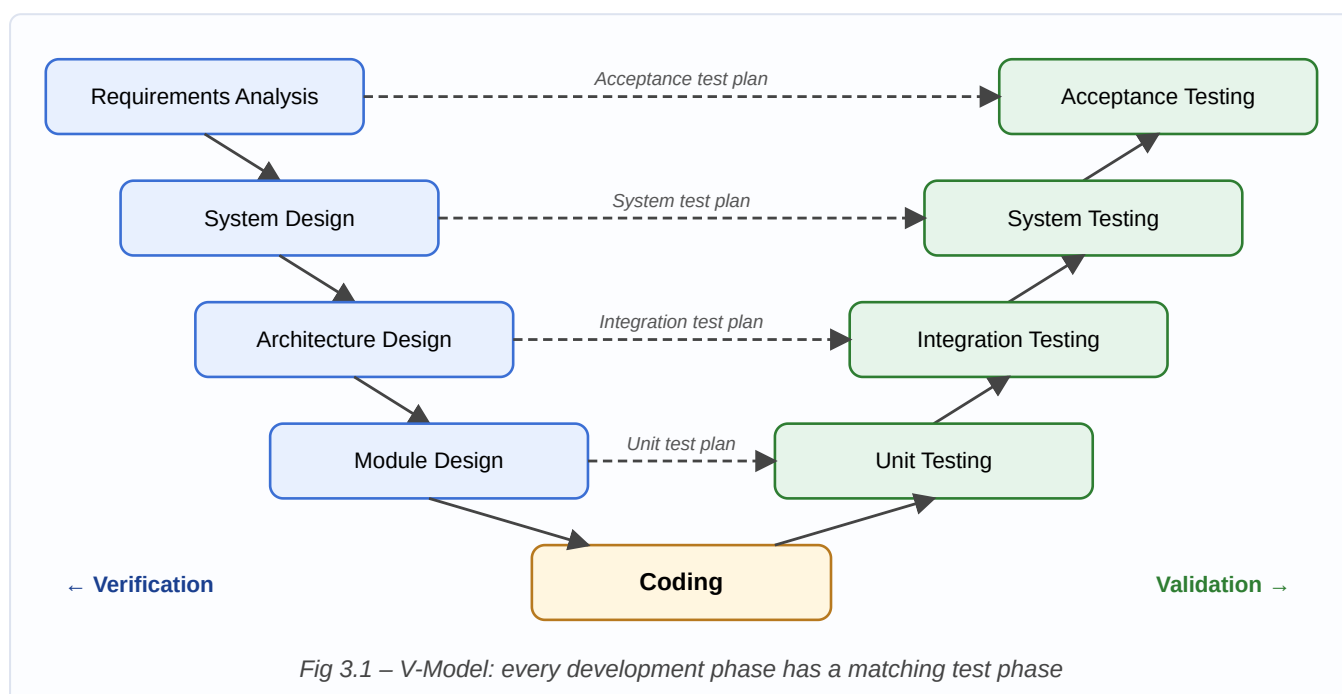
Testing in each phase of the SDLC

Testing is not a single phase at the end. It runs alongside development, because **the cost of fixing a defect grows sharply in later phases**.

SDLC phase	Testing / QA activity
Requirement analysis	Review the requirements (SRS) for completeness, ambiguity and testability. Build the requirement traceability matrix (RTM). Write the acceptance test plan .
System design	Review the design documents. Plan the system tests and decide on the test environment and tools.
Architecture / high-level design	Check module interfaces and data flow. Write the integration test plan .
Detailed / module design	Review the logic of each module. Write unit test cases .
Coding	Do code reviews, walkthroughs and static analysis. Developers run unit tests .
Testing	Run integration, system, regression and performance tests. Log and track defects.
Deployment	Do acceptance testing (alpha/beta, UAT), installation testing and smoke tests in production.
Maintenance	Run regression tests after every change, and confirmation tests for fixes.

V-Model (Verification and Validation model)

- The V-Model is an extension of the waterfall model.
- **Each development phase on the left arm has a matching testing phase on the right arm.**
- Test planning for a level starts as soon as the matching development phase is finished.



- **Left arm (Verification):** "Are we building the product right?" These are static activities such as reviews of the requirements and designs.
- **Bottom (Coding):** the point where the two arms meet.
- **Right arm (Validation):** "Are we building the right product?" These are dynamic activities that execute the code.

Development phase	Matching test phase	What it checks
Requirements analysis	Acceptance testing	The user's business needs are met
System design	System testing	The whole system against its specification
Architecture design	Integration testing	Interfaces between modules
Module design	Unit testing	Each module on its own

Advantages

- Tests are planned early, so defects are found early.
- It is simple and disciplined, and each phase has clear deliverables.
- It works well for small projects with stable requirements.

Disadvantages

- It is rigid and handles changing requirements poorly.
- No working software is produced until late in the project.
- There is no prototype, and risk analysis is limited.

Q4. Software Review, Inspection and Walkthrough; Verification vs Validation

Software review

- A **review** is a *static testing* technique. A work product (requirements, design, code, test plan) is examined by people **without executing it**, in order to find defects early.
- Types of review, from least formal to most formal:
 1. Informal review
 2. Walkthrough
 3. Technical review
 4. Inspection
- **Benefits:**
 - defects are found early, when they are cheap to fix,
 - productivity goes up,
 - testing time and cost go down,
 - knowledge is shared across the team.

Inspection

- An inspection is the **most formal** type of review. It was introduced by **Michael Fagan** at IBM.
- It is led by a trained **moderator** and uses **checklists, entry/exit criteria and metrics**.

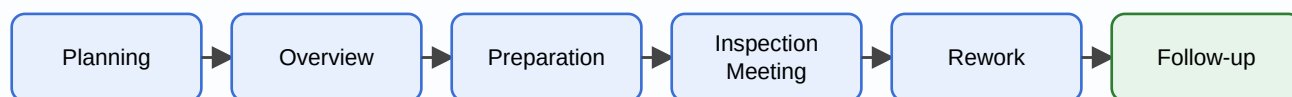


Fig 4.1 – Steps of a formal inspection (Fagan inspection)

Roles in an inspection

Role	Responsibility
Moderator	Leads the inspection and plans the meeting.
Author	Wrote the work product and later fixes the defects.
Reader	Reads the work product aloud during the meeting.
Recorder / Scribe	Writes down the defects that are found.
Inspectors	Look for defects, using checklists.

Steps

1. **Planning:** the moderator selects the team and sets the schedule.
2. **Overview:** the author explains the work product to the team.
3. **Preparation:** each inspector studies the material alone.
4. **Inspection meeting:** the team logs defects. Defects are found here, not solved.
5. **Rework:** the author fixes the defects.
6. **Follow-up:** the moderator checks that every defect has been fixed.

Walkthrough

- A walkthrough is a semi-formal review that is **led by the author**.
- The author "walks" the participants through the document or code, often by stepping through scenarios or sample inputs.
- **Main aims:** knowledge transfer, getting feedback and building a common understanding.
- Preparation is optional, and formal metrics are usually not collected.

Basis	Walkthrough	Inspection
Led by	Author	Trained moderator
Formality	Semi-formal / informal	Most formal
Preparation	Optional	Mandatory
Checklists and metrics	Rarely	Always
Purpose	Understanding, knowledge sharing	Finding as many defects as possible
Recorder	Optional	Mandatory

Verification vs Validation

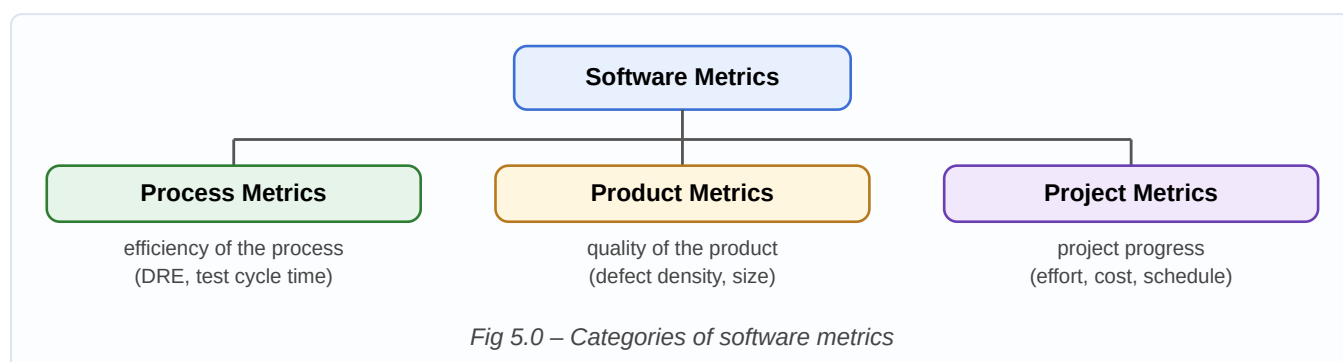
Basis	Verification	Validation
Question	<i>"Are we building the product right?"</i>	<i>"Are we building the right product?"</i>
Type	Static, no code is executed	Dynamic, code is executed
Methods	Reviews, walkthroughs, inspections, desk-checking	Black-box and white-box testing, UAT
Checks against	Specifications and design documents	User needs and requirements
Done by	QA team, peers	Testing team, users
Timing	Comes first, during every phase	Comes after verification, on working code
Finds	Defects early, which is cheaper	Failures in the running product
V-Model side	Left arm	Right arm

Q5. Software Testing Metrics, their types and the metrics life cycle

What is a metric?

- A **metric** is a quantitative measure of the degree to which a system, component or process has a given attribute.
- Testing metrics are used to:
 - estimate the quality and progress of testing,
 - make decisions about the next phase (for example, whether to release),
 - improve processes and predict effort.
- As the saying goes, "You cannot control what you cannot measure."

Types / categories



Category	Measures	Examples
Process metrics	How effective and efficient the process is	Defect Removal Efficiency, test execution productivity, review effectiveness
Product metrics	Characteristics of the product	Size (LOC, function points), complexity, defect density, reliability
Project metrics	Project management	Effort, cost, schedule variance, number of testers

Metrics are also classified as:

- **Manual vs Automation metrics.**
- **Base (direct) metrics:** raw counts collected directly. Examples: number of test cases written, executed, passed, failed and blocked; number of defects.
- **Calculated (derived) metrics:** computed from base metrics. Examples:

Metric	Formula
Test case execution %	$(\text{No. of test cases executed} \div \text{Total test cases}) \times 100$
Test case pass %	$(\text{No. of passed} \div \text{No. executed}) \times 100$
Defect density	$\text{No. of defects} \div \text{Size (KLOC or FP)}$
Defect Removal Efficiency (DRE)	$E \div (E + D) \times 100$, where E = defects found before release and D = defects found after release
Defect leakage %	$(\text{Defects found in UAT/production} \div \text{Defects found in testing}) \times 100$

Metric	Formula
Test effectiveness	$(\text{Defects found by testing} \div \text{Total defects}) \times 100$
Defect rejection ratio	$(\text{Rejected defects} \div \text{Total defects reported}) \times 100$

Worked example of DRE: 90 defects were found during testing and 10 were found by customers after release. $\text{DRE} = 90 \div (90 + 10) \times 100 = 90 \%$.

Metrics life cycle

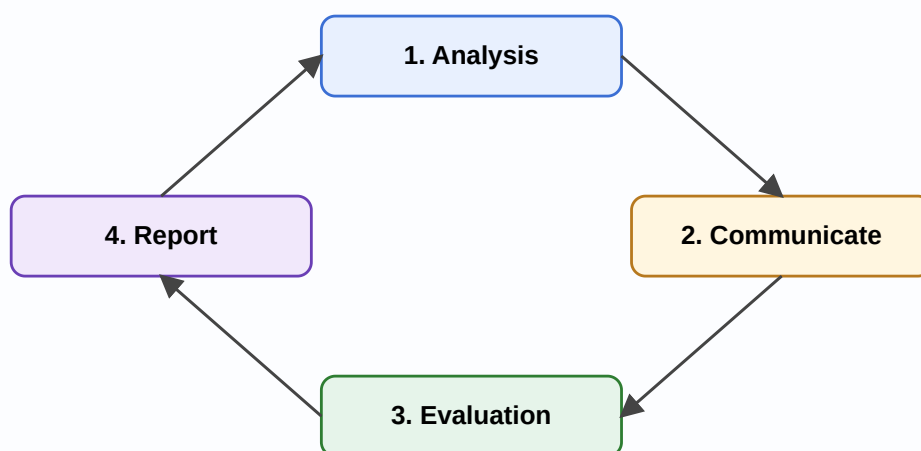


Fig 5.1 – Software testing metrics life cycle

1. Analysis

- Identify the metrics to collect, guided by the goal.
- Define each metric clearly.
- Decide its parameters and how it will be measured.

2. Communicate

- Explain the need for the metric to stakeholders and the testing team.
- Tell the team what data has to be captured and how.

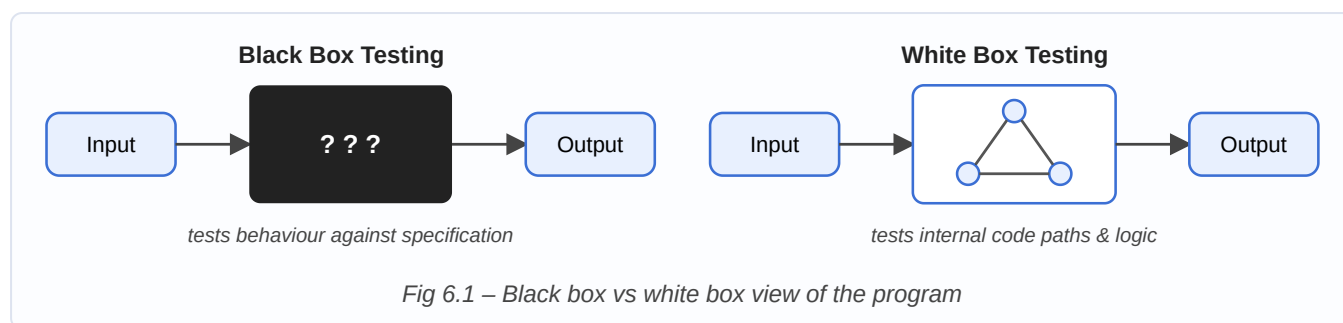
3. Evaluation

- Capture and verify the data.
- Calculate the metric values from the data.

4. Report

- Write a report with a clear conclusion.
- Share it with stakeholders and get their feedback.
- Use the feedback to improve. The cycle then repeats.

Q6. Black Box vs White Box testing and their techniques



Basis	Black Box Testing	White Box Testing
Other name	Behavioural, functional, closed-box	Structural, glass-box, clear-box
Knowledge of code	Not needed	Needed
Based on	Requirements / SRS	Source code and design
Done by	Testers, end users	Usually developers
Levels	System and acceptance testing	Unit and integration testing
Finds	Missing functions, interface errors, behaviour errors	Logic errors, dead code, untested paths
Techniques	BVA, equivalence partitioning, decision table, state transition, use-case	Statement, branch, condition, basis path, loop, data-flow testing
Time	Less time-consuming	More time-consuming

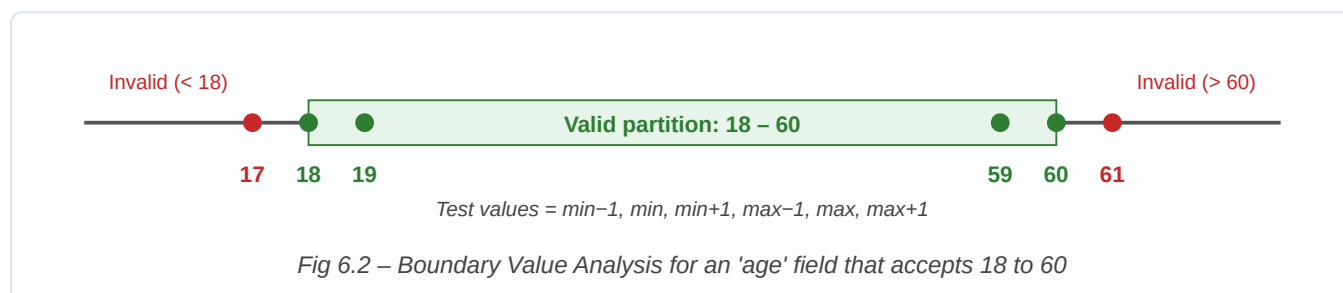
Black box techniques

1. Equivalence Partitioning (EP)

- Split the inputs into classes that the program should treat the same way.
- Test **one value from each class**.
- Example: age 18–60 gives three classes: below 18 (invalid), 18 to 60 (valid) and above 60 (invalid).

2. Boundary Value Analysis (BVA)

- Errors cluster at the **edges** of input ranges.
- Test the values at each edge and just inside and outside it.



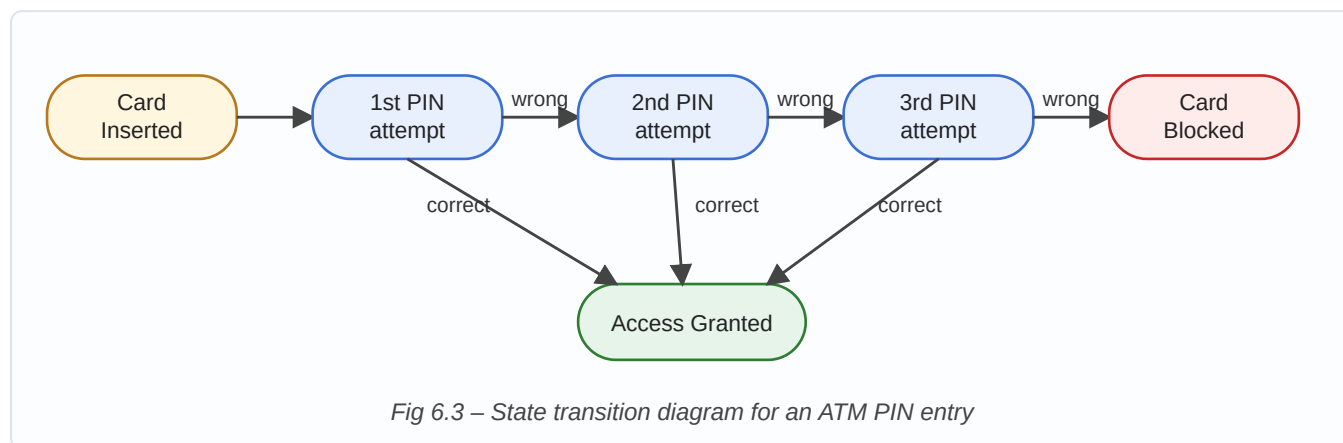
For a range [min, max], test min-1, min, min+1, max-1, max and max+1. For age 18–60 these are **17, 18, 19, 59, 60, 61**.

3. Decision Table testing

- Make a table of every combination of conditions and the action expected for each one.
- Useful for complex business rules.

4. State Transition testing

- Used when the output depends on the **current state** as well as the input.
- The system is modelled as states, events (inputs) and transitions.
- Test cases cover every state and every transition, valid and invalid.



Example (ATM PIN):

Current state	Input	Next state
1st attempt	Correct PIN	Access granted
1st attempt	Wrong PIN	2nd attempt
2nd attempt	Wrong PIN	3rd attempt
3rd attempt	Wrong PIN	Card blocked

White box techniques

- **Statement coverage:** every statement runs at least once.
- **Branch / decision coverage:** every true and false outcome of each decision is taken at least once.
- **Condition coverage:** every individual condition is evaluated as both true and false.
- **Basis path testing** (McCabe):
 1. Draw the control flow graph.
 2. Compute the **cyclomatic complexity V(G)**.
 3. Find V(G) **independent paths**.
 4. Write one test case for each path. - This guarantees that every statement and every branch is executed at least once (see Q11 for a full example).
- **Loop testing:** for each loop, run it 0 times, 1 time, 2 times, a typical number of times, and max-1, max and max+1 times.
- **Data-flow testing:** follow each variable from its definition to where it is used.

Q7. Integration testing approaches: Top-down and Bottom-up

What is integration testing?

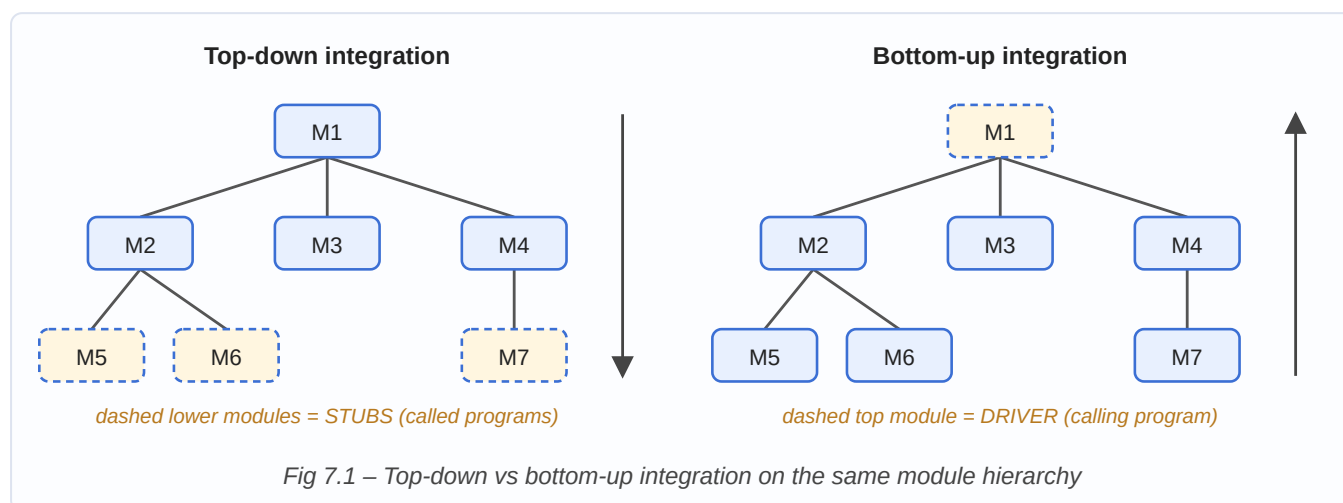
- Unit-tested modules are combined and tested as a group.
- The goal is to find **interface defects**: wrong parameters, data loss between modules, and timing problems.

Approaches

- **Big-bang**: all modules are combined at once.
- **Incremental**: modules are added one at a time. There are three kinds:
 - top-down,
 - bottom-up,
 - sandwich (hybrid).

Two kinds of dummy programs are used:

- **Stub**: a dummy *called* module. It stands in for a lower-level module that is not ready yet, and is used in top-down integration.
- **Driver**: a dummy *calling* module. It stands in for a higher-level module, calls the module under test and passes it data. It is used in bottom-up integration.



Top-down integration

1. Testing starts at the **main control module (M1)**. Stubs replace all the modules directly below it.
2. Stubs are replaced by real modules one at a time, moving downward. The order can be:
 - **depth-first**: M1 → M2 → M5 → M6 → M3 → M4 → M7, or
 - **breadth-first**: M1 → M2 → M3 → M4 → M5 → M6 → M7.
3. Tests are run after each module is added. **Regression testing** checks that nothing that worked before is now broken.

Advantages

- Major control and decision points are tested early.
- An early skeleton or prototype of the system is available to show users.

- No drivers are needed.

Disadvantages

- Many stubs are needed, and writing them is hard.
- Lower-level modules are tested late and less thoroughly.

Bottom-up integration

1. The lowest-level modules (M5, M6, M7) are combined into **clusters (builds)** that perform a sub-function.
2. A **driver** is written for each cluster to control test input and output.
3. Each cluster is tested.
4. Drivers are removed and clusters are combined moving upward until the main module M1 is reached.

Advantages

- No stubs are needed.
- Low-level utility modules, which are often critical, are tested thoroughly.
- Test conditions are easier to create.

Disadvantages

- The program as a whole does not exist until the last module is added.
- Top-level design and control flaws are found late.
- Drivers have to be written.

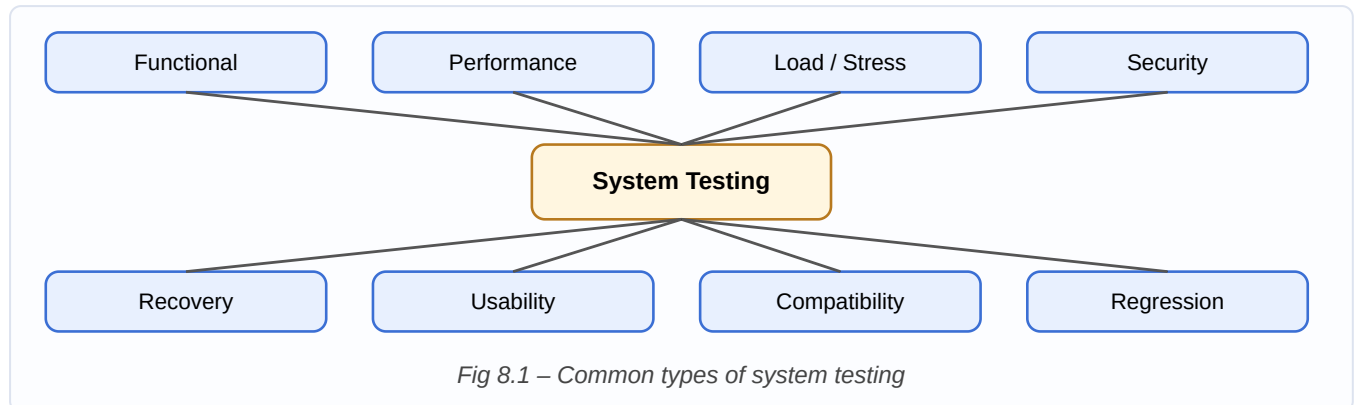
Basis	Top-down	Bottom-up
Starts from	Main (top) module	Lowest-level modules
Dummy module	Stubs	Drivers
Early prototype	Yes	No
Finds design errors	Early	Late
Best for	Systems where top-level control matters most	Systems built from reusable low-level components

Sandwich (hybrid) integration combines both: top-down for the upper levels and bottom-up for the lower levels, meeting at a middle layer.

Q8. System testing, its types, and Alpha and Beta testing

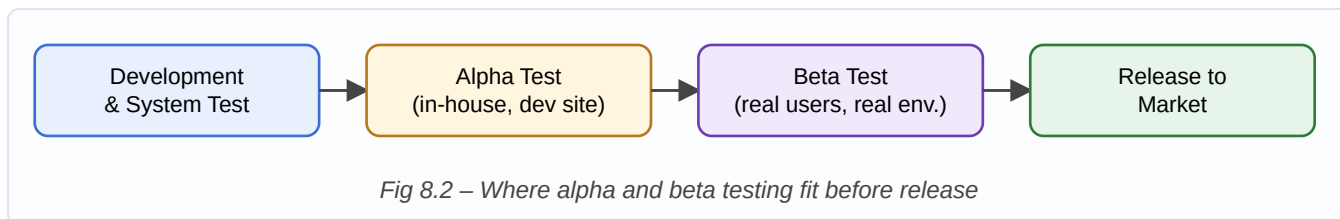
What is system testing?

- System testing tests the **complete, integrated system** against its **specified requirements** (the SRS), both functional and non-functional.
- It is **black-box** testing.
- It is done by an independent test team, in an environment as close to production as possible.
- It comes **after integration testing and before acceptance testing**.



Type	Purpose
Functional testing	Every function works as described in the SRS.
Performance testing	Response time, throughput and resource use under expected load.
Load testing	Behaviour at the expected peak number of users or transactions.
Stress testing	Behaviour beyond the limits (abnormal load). Finds the breaking point and checks that the system recovers gracefully.
Security testing	Protection against unauthorised access. Checks confidentiality, integrity and authentication.
Recovery testing	The software is forced to fail (crash, power loss), then the test checks that it recovers properly and data is intact.
Usability testing	How easy the system is to learn and use, and how consistent its UI is.
Compatibility testing	The system works on different operating systems, browsers, devices and networks.
Regression testing	Changes or fixes have not broken features that already worked.
Installation testing	Install, upgrade and uninstall work correctly.
Volume testing	Behaviour with very large amounts of data.

Alpha and Beta testing (types of acceptance testing)



Alpha testing

- Done at the **developer's site** by internal staff or potential users, with developers watching.
- It takes place in a controlled lab environment, before the product is released to outside users.

Beta testing

- Done at the **customer's site** by **real end users**, in their real environment.
- Developers are not present.
- Users report problems back. This is also called *field testing*, as in "beta versions" of apps.

Basis	Alpha testing	Beta testing
Location	Developer's site	User's / customer's site
Performed by	Internal testers, staff, selected users	Real end users
Environment	Controlled (lab)	Uncontrolled (real world)
Developer present	Yes	No
Testing type	White box and black box	Black box only
Duration	Long, many cycles	Short, a few weeks
Issues found	Bugs, crashes, missing features	Usability, compatibility, real-use feedback
Comes	First	After alpha, just before release

Q9. Smoke testing and its benefits

Definition

- **Smoke testing**, also called **Build Verification Testing (BVT)**, is a quick, shallow set of tests run on every new build.
- It checks that the **most important functions work** and that the build is **stable enough** for detailed testing.
- The name comes from hardware testing: switch the device on, and if it does not catch fire (smoke), carry on testing.
- It answers one question: *"Is the build testable at all?"*

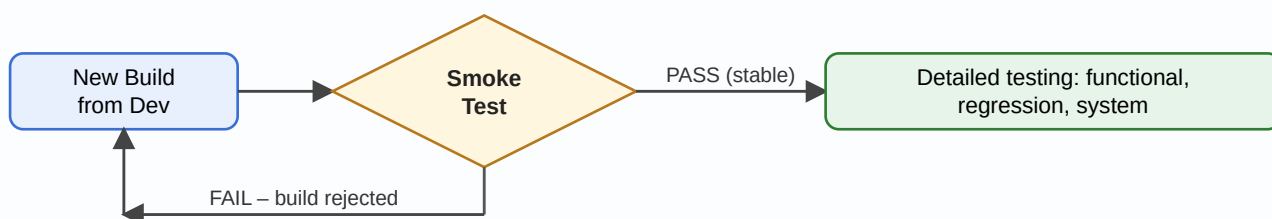


Fig 9.1 – Smoke test as the gate for every new build

Characteristics

- It covers the main end-to-end flows **broadly but shallowly**. Example for an e-commerce site: launch the app → log in → search → add to cart → check out.
- It is run on **every new build**, usually daily, and is often automated.
- If it fails, the build is **rejected** and returned to the developers. No time is spent on detailed testing.
- It is usually done by testers. Developers can also run it before handing over the build.

Smoke testing is often confused with sanity testing:

Smoke testing	Sanity testing
Checks the stability of the whole build	Checks that a specific fix or change works
Broad and shallow	Narrow and deep
Done on early or unstable builds	Done on relatively stable builds
Usually scripted or automated	Usually unscripted

Benefits of smoke testing

1. **Finds integration and serious defects early**, as soon as a build is made.
2. **Saves time and effort**, because testers do not waste a full test cycle on a broken build.
3. **Lowens integration risk**. Daily builds plus a smoke test catch problems while the changes are still small.
4. **Gives quick feedback** to developers about the health of the build.
5. **Improves quality and confidence**, because only stable builds move on to further testing.
6. **Easy to automate** and to run in CI/CD pipelines.

7. **Makes progress easier to see**, because each day's build is shown to be working.

Q10. Cause and effect diagram, Pareto diagram and Scatter diagram

These are three of the **seven basic quality-control tools**. The others are check sheet, histogram, control chart and stratification or flowchart.

1. Cause and effect diagram (Ishikawa / fishbone diagram)

- The cause and effect diagram was developed by **Kaoru Ishikawa**.
- It shows **all possible causes** of a problem (the effect) in an organised way, so that the **root cause** can be found.
- The effect is written at the "head" of the fish. The main categories of causes form the "bones".

Common categories are:

- **6 M's**: Man, Machine, Method, Material, Measurement, Mother-nature (environment).
- For software: People, Process, Tools, Requirements, Environment, Methods.

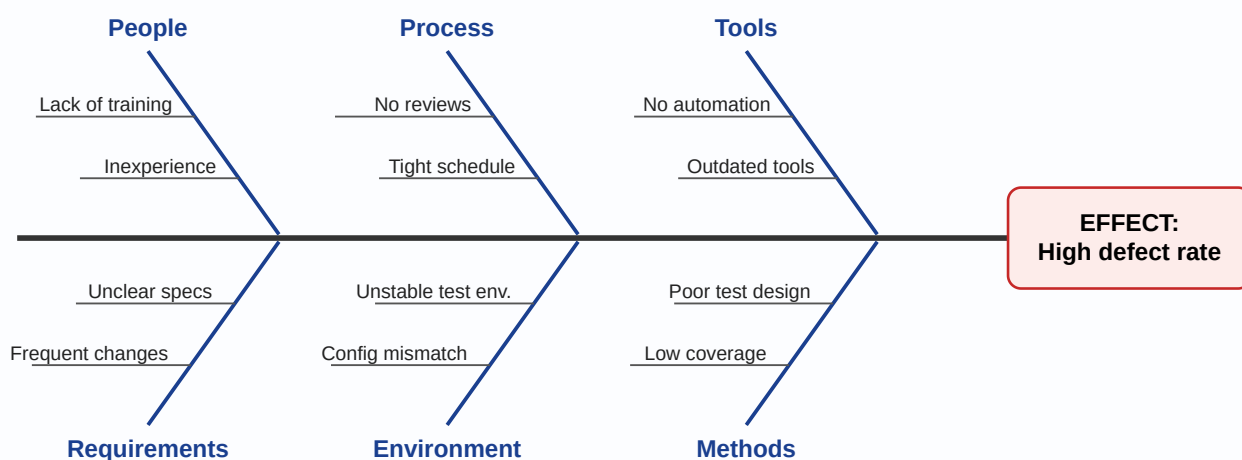


Fig 10.1 – Cause and effect (Ishikawa / fishbone) diagram

Steps to draw it

1. Write the problem (effect) in a box on the right and draw the main horizontal spine.
2. Decide the main cause categories and draw them as bones joined to the spine.
3. Brainstorm causes for each category and attach them as smaller branches.
4. Keep asking "Why?" to break each cause down into sub-causes.
5. Analyse the diagram and pick out the most likely root causes to check.

Uses: root-cause analysis, team brainstorming, deciding where to focus improvement.

2. Pareto diagram

- A Pareto diagram is a **bar chart with the bars sorted in descending order** of frequency, plus a **cumulative-percentage line**.
- It is based on the **Pareto principle (80/20 rule)**: about **80 % of problems come from 20 % of the causes**.
- It shows the "vital few" causes to fix first, as opposed to the "trivial many".

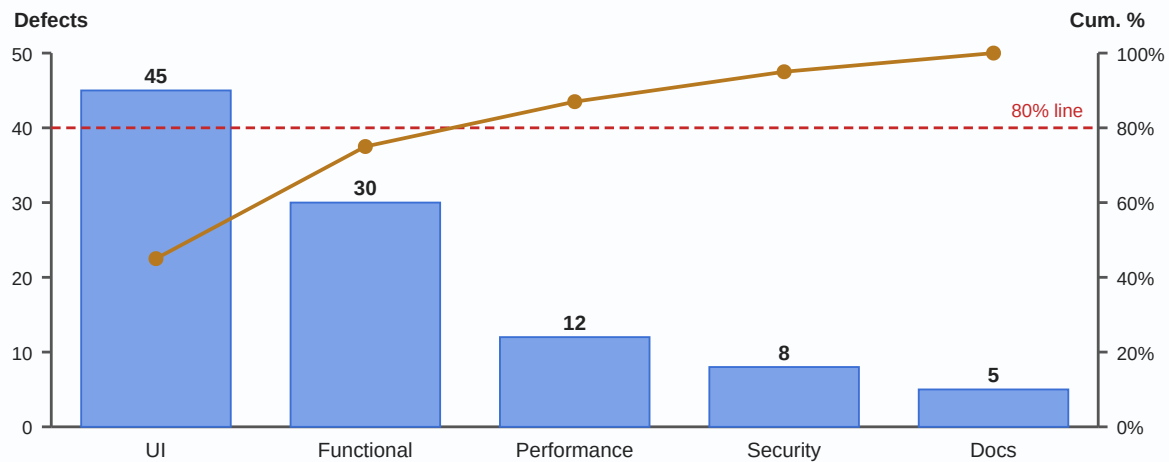


Fig 10.2 – Pareto diagram of defects by category (bars) with cumulative % line

Steps to draw a Pareto diagram

1. Decide the problem and the categories of causes to compare, for example defect types.
2. Choose the measure (frequency or cost) and the time period.
3. Collect the data, for example using a check sheet.
4. Sort the categories **from largest to smallest**.
5. Calculate the **percentage** of each category and the **cumulative percentage**.
6. Draw the left vertical axis (frequency) and the right vertical axis (0–100 %). Draw the horizontal axis with the categories.
7. Draw the bars in descending order.
8. Plot the cumulative percentages and join them into a line.
9. Draw a line at 80 %. The categories to the left of where it meets the curve are the vital few.

In the example: UI (45 %) and Functional (30 %) defects together make **75 %** of all defects. Fixing these two categories first gives the biggest gain.

3. Scatter diagram

- A scatter diagram plots **pairs of numeric data (X, Y)** as points, to show whether two variables are **related**.
- Example: module size and number of defects.
- Possible patterns:
 - positive correlation (Y rises as X rises),
 - negative correlation (Y falls as X rises),
 - no correlation (random cloud),
 - non-linear relationship.

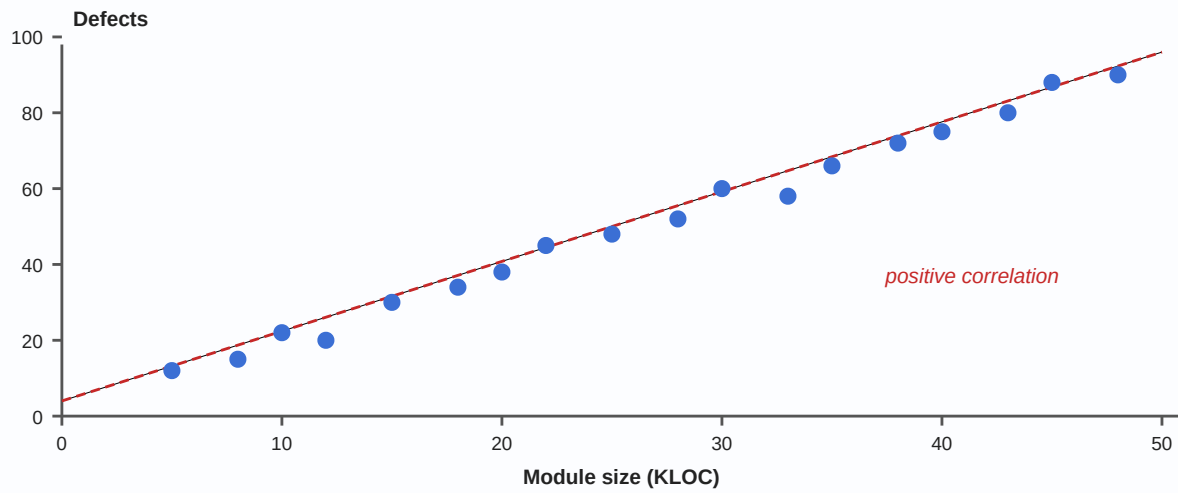


Fig 10.3 – Scatter diagram: module size vs number of defects

Steps

1. Collect pairs of data for the two variables (at least 20–30 pairs).
2. Put the suspected cause on the X-axis and the effect on the Y-axis.
3. Plot each pair as a point.
4. Look at the pattern, and draw a trend line if needed.

Note: correlation does not prove causation.

Uses: testing a suspected cause from the fishbone diagram, and estimating effort or defects from size.

Q11. Cyclomatic Complexity with an example

Definition

- **Cyclomatic complexity $V(G)$** was proposed by **Thomas McCabe (1976)**.
- It is a software metric that measures the **logical complexity** of a program.
- It equals the **number of linearly independent paths** through the program's control flow graph (CFG).
- This gives:
 - the **minimum number of test cases** needed for basis path testing, which guarantees full statement and branch coverage,
 - an indication of how hard the code is to understand, test and maintain.

Three ways to calculate it

1. $V(G) = E - N + 2$, where E = number of edges and N = number of nodes.
2. $V(G) = P + 1$, where P = number of predicate (decision) nodes.
3. $V(G)$ = **number of regions** of the flow graph, counting the outside region.

$V(G)$	Risk
1 – 10	Simple program, low risk
11 – 20	More complex, moderate risk
21 – 50	Complex, high risk
> 50	Untestable, very high risk

Example

```

1  i = 1;
2  while (i <= n) {
3      if (i % 2 == 0)
4          print("even");
5      else
6          print("odd");
7      i = i + 1;
8  }
9  return;
```

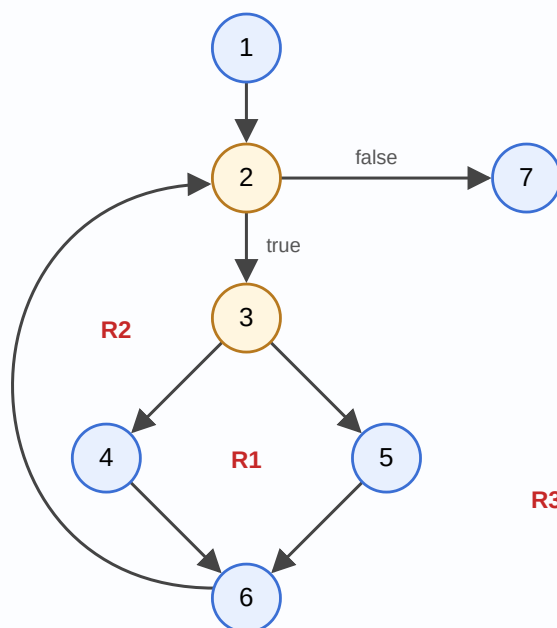


Fig 11.1 – Control flow graph (shaded nodes = predicate nodes, R = regions)

Edges: $1 \rightarrow 2$, $2 \rightarrow 3$, $2 \rightarrow 7$, $3 \rightarrow 4$, $3 \rightarrow 5$, $4 \rightarrow 6$, $5 \rightarrow 6$, $6 \rightarrow 2$. That is **$E = 8$** . **Nodes:** 1 to 7. That is **$N = 7$** .

Predicate nodes: node 2 (while) and node 3 (if). That is **$P = 2$** .

Method	Calculation	$V(G)$
$E - N + 2$	$8 - 7 + 2$	3
$P + 1$	$2 + 1$	3
Regions	R1, R2, R3	3

Independent paths, which give the 3 test cases:

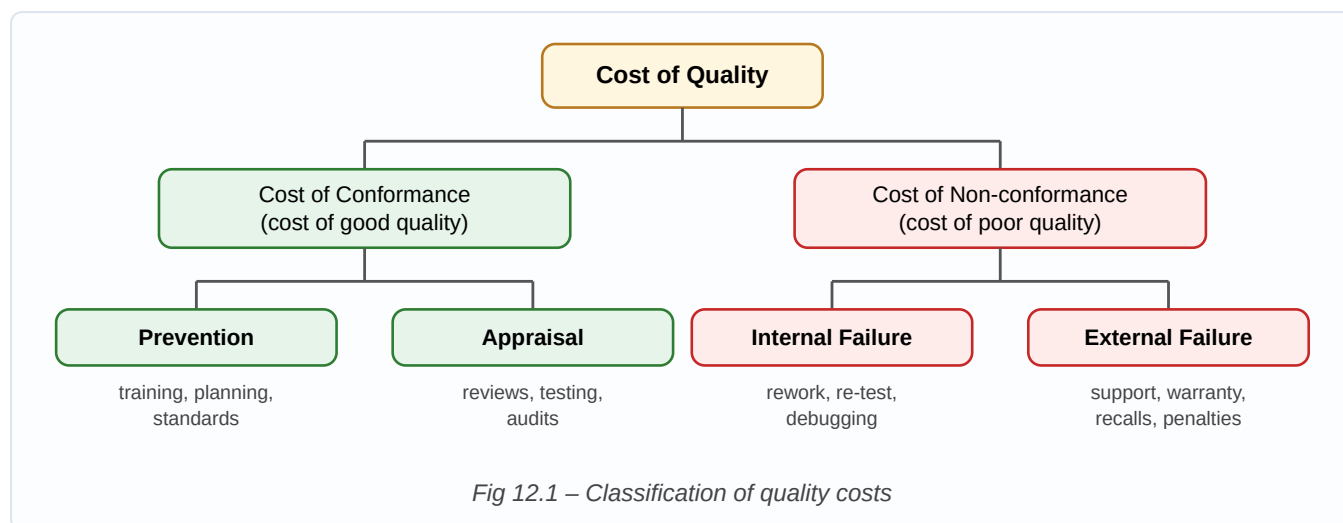
Path	Route	Test input
Path 1	$1 \rightarrow 2 \rightarrow 7$	$n = 0$ (loop not entered)
Path 2	$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6 \rightarrow 2 \rightarrow 7$	$n = 2$ (the pass with $i = 2$ takes the even branch)
Path 3	$1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 2 \rightarrow 7$	$n = 1$ ($i = 1$ is odd)

So **3 test cases** are enough to execute every statement and every branch at least once.

Q12. Quality Costs: types and how to measure them

Definition

- **Cost of Quality (CoQ)** is the total cost of everything an organisation spends to **achieve quality**, plus everything it loses because of **poor quality**.
- Philip Crosby: "Quality is free. It's not quality that's expensive, but the lack of it."
- In short: **CoQ = Cost of Conformance + Cost of Non-conformance**.



Types of quality costs

Type	Meaning	Software examples
1. Prevention costs	Money spent so that defects do not happen	Training, quality planning, process improvement, standards, formal technical reviews of plans, tool selection
2. Appraisal costs	Money spent to find defects by measuring and checking the product	Reviews and inspections, testing (unit to acceptance), test tools and environment, audits, equipment calibration
3. Internal failure costs	Cost of defects found before the product reaches the customer	Rework, re-testing, debugging, scrap, failure analysis, schedule delays
4. External failure costs	Cost of defects found after release, by the customer	Help-desk and support, patches, warranty, refunds, penalties and lawsuits, lost customers and reputation

- Prevention and appraisal costs form the **cost of conformance**.
- Internal and external failure costs form the **cost of non-conformance**.
- The cost of fixing a defect **rises steeply** from prevention, to appraisal, to internal failure, to external failure.
- Spending a little more on prevention and appraisal greatly reduces failure costs.

How to measure quality costs

1. **Identify cost elements.** List the activities under each of the four categories for your organisation.
2. **Collect the data.** Use time sheets (effort spent on reviews, testing and rework), defect-tracking systems, help-desk logs, warranty and refund records, and finance records.

3. **Convert to money.** Multiply effort hours by the cost rate, then add tool, licence, equipment and penalty costs.

4. **Add up each category:**

- Prevention cost = Σ (prevention activities)
- Appraisal cost = Σ (appraisal activities)
- Internal failure cost = Σ (rework, re-test and so on before release)
- External failure cost = Σ (support, fixes, penalties and so on after release)
- **Total CoQ** = Prevention + Appraisal + Internal failure + External failure

5. **Normalise and compare.** Express CoQ as a **percentage of sales or of project cost**, or as cost per defect. Track it over time and against benchmarks.

6. **Analyse.** A Pareto analysis of the failure costs shows where more prevention would pay off most.

Example: Prevention ₹1 L, Appraisal ₹3 L, Internal failure ₹4 L, External failure ₹2 L.

- Total CoQ = **₹10 L**.
- Of this, ₹4 L is the cost of good quality and ₹6 L is the cost of poor quality.
- The goal is to shift spending towards prevention.

Q13. Software reliability: elements, factors and metrics

Definition

- **Software reliability** is the probability that the software will work without failure, in a specified environment, for a specified period of time (IEEE).
- Example: a reliability of 0.96 over 100 hours means that in 96 out of 100 such periods the program runs without failure.

Elements of software reliability

1. **Fault avoidance (prevention):** stop faults from entering the system, using good requirements, design methods, standards and reviews.
2. **Fault removal (detection):** find and remove faults, using verification, validation and testing.
3. **Fault tolerance:** keep the system working even when faults are present, using redundancy, exception handling, checkpoints and recovery.
4. **Fault forecasting:** estimate how many faults remain and how reliable the system will be in future, using reliability models and failure data.

Related attributes:

- **Availability:** the system is ready when needed.
- **Maintainability:** the system is easy to repair.
- **Safety** and **security**.

Factors affecting software reliability

Factor	Effect
Size and complexity of the code	More code and higher complexity mean more chances of faults.
Quality of requirements and design	Unclear or changing requirements lead to defects.
Development process maturity	Use of reviews, standards and configuration management.
Testing effort and coverage	More thorough testing removes more faults.
Skill and experience of the team	Human errors are the main source of defects.
Operational profile / environment	How users actually use the system, and the hardware and OS it runs on.
Fault-tolerance mechanisms	Exception handling, redundancy and recovery.
Number of changes / maintenance	Every change can bring in new faults.
Tools and technology used	Language, compilers, libraries.

Reliability metrics

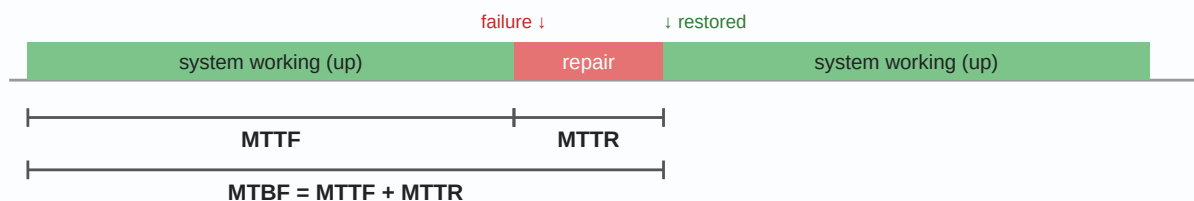


Fig 13.1 – Relationship between MTTF, MTTR and MTBF

Metric	Formula / meaning
MTTF (Mean Time To Failure)	Average time the system runs between being restored and its next failure
MTTR (Mean Time To Repair)	Average time needed to find and fix a fault after a failure
MTBF (Mean Time Between Failures)	$MTBF = MTTF + MTTR$. Example: $MTTF\ 200\ h + MTTR\ 4\ h = MTBF\ 204\ h$.
ROCOF (Rate of Occurrence of Failure)	Number of failures per unit time. Example: 2 failures per 100 operational hours.
POFOD (Probability of Failure on Demand)	Probability that the system fails when a request is made. Example: POFOD 0.001 means 1 failure in 1000 requests. Used for safety systems.
Availability (AVAIL)	$MTTF \div (MTTF + MTTR) \times 100\ %$, the share of time the system is usable. Example: $200 \div 204 = 98\ %$.

Reliability metrics are also grouped by what they measure:

- **Product metrics:** size (KLOC, function points), complexity, test coverage.
- **Project management metrics:** good management leads to better products.
- **Process metrics:** process quality affects reliability, for example DRE.
- **Fault and failure metrics:** number of faults found in testing and failures reported by users, used to calculate MTBF and similar measures.

Q14. Guidelines for Formal Technical Reviews (FTR)

What is an FTR?

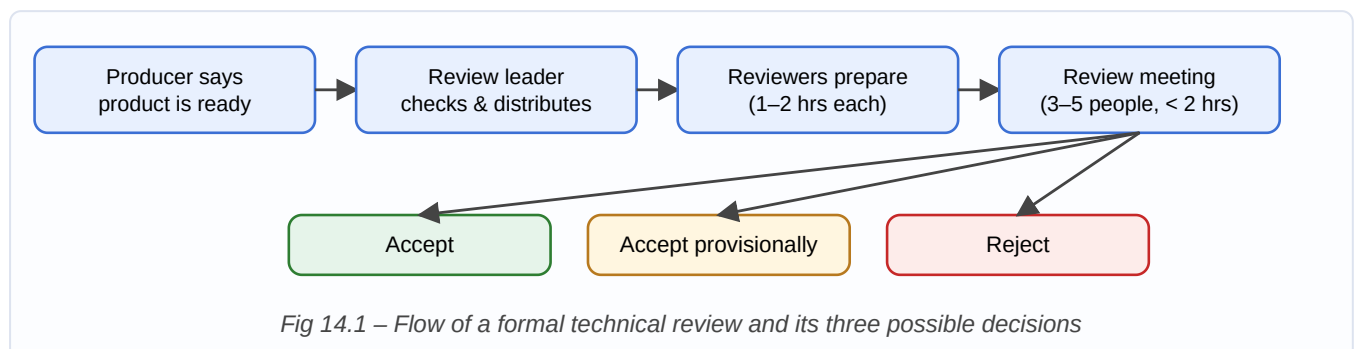
- A **Formal Technical Review** is a software quality-control activity carried out by software engineers.
- It is a **class of reviews** that includes walkthroughs and inspections.

Objectives

1. Find errors in function, logic or implementation.
2. Check that the software meets its requirements.
3. Make sure it follows the agreed standards.
4. Make development more uniform.
5. Make projects easier to manage.

Review meeting constraints (Pressman)

- **3 to 5 people** should take part.
- Each reviewer should prepare in advance, for **no more than 2 hours**.
- The meeting itself should last **less than 2 hours**.
- Because of these limits, review a **small, specific part** of the software, not the whole system at once.



Guidelines for conducting an FTR

1. **Review the product, not the producer.** Point out errors politely. The meeting should be relaxed and constructive and should never embarrass or blame the author.
2. **Set an agenda and keep to it.** The review leader must keep the meeting on track and stop it from drifting.
3. **Limit debate and rebuttal.** If an issue cannot be settled quickly, record it for discussion outside the meeting.
4. **Point out problem areas, but do not try to solve every problem.** A review finds problems. Solutions are worked out later by the author or a small group.
5. **Take written notes.** A recorder writes down all issues, preferably where everyone can see them, so that the wording and priority can be agreed.
6. **Limit the number of participants and insist on advance preparation.** Keep the group small. Every reviewer must study the material before the meeting.
7. **Develop a checklist for each type of product reviewed.** Checklists for requirements, design, code and test documents help reviewers focus on the important issues.
8. **Allocate resources and schedule time for FTRs.** Reviews must be a planned task in the project schedule, including time for the rework that follows.

9. **Train all reviewers.** Training should cover both the technical process and the human side of reviewing.
10. **Review your early reviews.** Look back at how the first reviews went, so the review process itself can be improved.

Outcome of an FTR

At the end, all attendees decide to:

- **Accept** the product without further changes,
- **Reject** it because of severe errors (it is reviewed again after correction), or
- **Accept it provisionally**, where minor errors must be fixed but no further review is needed.

The **review summary report** records:

- what was reviewed,
- who reviewed it,
- the findings and conclusions.

An **issues list** is also produced and is used to track the corrections.

Q15. Six Sigma (DMAIC) and ISO 9000 standards

What is Six Sigma?

- **Six Sigma** is a data-driven quality-management method. It was started by **Motorola (1986)** and made popular by **GE**.
- It aims to **reduce variation and defects** in a process until it produces no more than **3.4 defects per million opportunities (DPMO)**. This equals 99.99966 % defect-free output.
- **σ (sigma)** is the standard deviation. "Six Sigma" means the specification limits are six standard deviations from the process mean.
- It relies on statistics, a focus on the customer, and trained people:
 - **Champions** sponsor the improvement projects.
 - **Master Black Belts** coach the Black Belts.
 - **Black Belts** lead the projects.
 - **Green Belts** work on the projects part-time.

Basic steps: DMAIC (for improving an existing process)

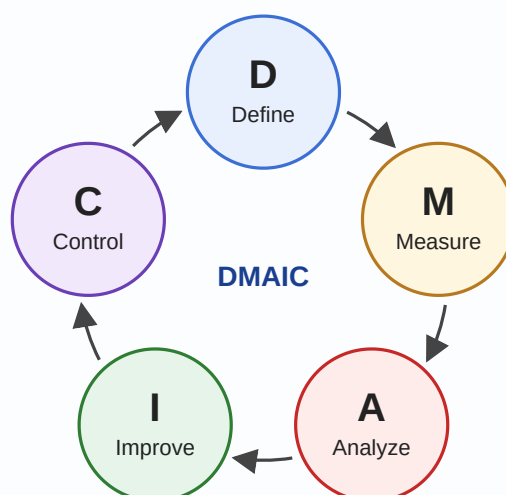


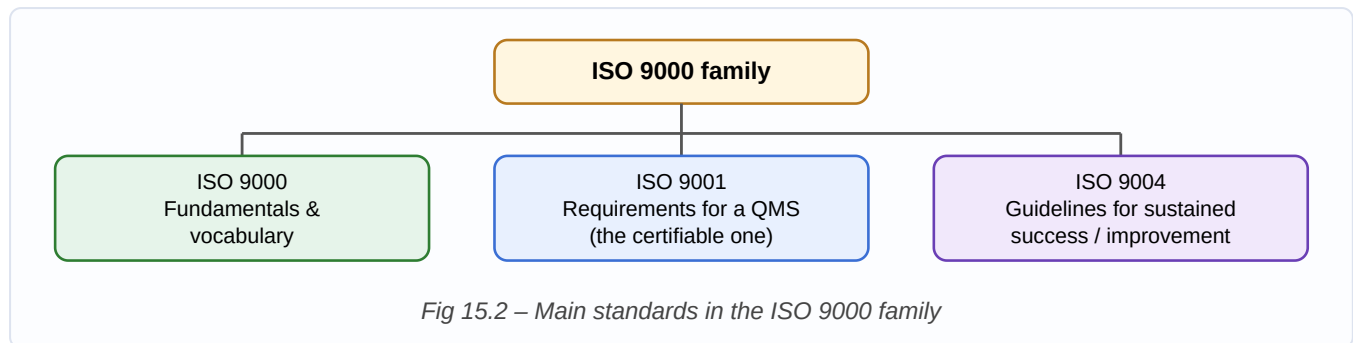
Fig 15.1 – The DMAIC improvement cycle of Six Sigma

Step	What is done	Tools
D – Define	Define the problem, the project goals and the customer requirements (CTQ: Critical To Quality). Set the scope and the team.	Project charter, SIPOC, voice of the customer
M – Measure	Measure current process performance and collect baseline data, for example the current defect rate.	Data collection, check sheets, DPMO, sigma level
A – Analyze	Analyse the data to find the root causes of defects and variation.	Fishbone diagram, Pareto chart, scatter diagram, 5 Whys, hypothesis tests
I – Improve	Develop, test and put in place solutions that remove the root causes.	Brainstorming, design of experiments, pilot runs
C – Control	Keep the gains: monitor the improved process, standardise it and document it.	Control charts, SOPs, audits

For **designing new processes or products**, Six Sigma uses **DMADV**: Define, Measure, Analyze, Design, Verify.

ISO 9000 standards

- **ISO 9000** is a family of international standards for **Quality Management Systems (QMS)**, published by the International Organization for Standardization.
- ISO 9000 **does not certify a product**. It certifies that the organisation has a **documented, consistently followed quality process**.



- **ISO 9000**: basic concepts and vocabulary.
- **ISO 9001**: the requirements an organisation must meet to be **certified**. It applies to software organisations too. **ISO 90003** gives guidelines for applying ISO 9001 to software.
- **ISO 9004**: guidance on improving performance and long-term success.

Quality management principles behind ISO 9000:

1. Customer focus
2. Leadership
3. Engagement of people
4. Process approach
5. Improvement
6. Evidence-based decision making
7. Relationship management

How certification works

1. Apply to a registrar.
2. The registrar does a pre-assessment.
3. The organisation documents its processes (quality manual, procedures).
4. The registrar runs a document review and an on-site audit.
5. If the audit passes, the certificate is issued.
6. Surveillance audits continue after that.

Advantages of ISO 9000

1. **Better product quality and consistency.** Processes are defined, documented and repeatable.
2. **More customer confidence and satisfaction.** Certification is recognised worldwide.
3. **Access to new markets.** Many clients and governments require ISO 9001 certification.
4. **Lower costs.** Less rework and waste, and fewer failures.
5. **Clear responsibilities.** Roles, procedures and records are documented.
6. **Continual improvement.** Internal audits and corrective and preventive actions drive improvement.

7. **Better staff morale and training.** People understand the processes and their own role.

8. **Decisions based on data,** from records and audits.

Limitations of ISO 9000

- It is document-heavy, and certification can be costly.
- It shows the process is followed, not that the product is excellent.
- It can become a "paperwork" exercise if it is not taken seriously.